

执行摘要

对项目进行综合评估，包括测试、性能分析、静态安全审查和架构重构。测试工程师应运行现有测试套件并测量测试覆盖率，找出失败的用例并补充测试，以提升可靠性和覆盖率。性能工程师需针对关键路径进行基准测试和内存分析，识别瓶颈并优化热点代码。静态分析/安全工程师应使用自动化扫描工具（如 SonarQube、CodeQL 等）检查代码质量和安全漏洞，及时修复高危问题【1 + L230-L239】【19 + L478-L482】。重构/架构工程师分析代码复杂度（如圈复杂度 Cyclomatic Complexity）和模块依赖，提出简化和解耦方案，以提高可维护性【22 + L150-L158】。DevOps 工程师负责完善文档和 CI/CD 流程，确保测试、构建和部署自动化运行【15 + L81-L85】。最终将生成可供模型调用的详细提示词文件，包含上下文说明、约束条件、输入输出示例、测试用例和回归测试流程。目标是提升性能（如响应时间、内存占用）、增加测试覆盖、消减漏洞数量、降低代码复杂度等指标（对比表见下表），并输出一份改进清单和时间计划甘特图。所有建议均基于对标最佳实践和权威资料（如 Atlassian 和微软文档所示）【11 + L649-L657】【22 + L203-L210】。

测试工程师任务提示词

- **目标：**执行并改进现有测试套件，提高测试覆盖率并确保所有关键功能通过测试。
- **输入：**代码库（源文件、测试文件）、现有测试运行命令（如 `pytest` , `mvn test` , `dotnet test` ）、示例配置（如 `pytest.ini` , `pom.xml` , CI 配置片段）。
- **具体步骤：**
 - 使用对应测试框架命令（如 `pytest --maxfail=1 --disable-warnings` , `mvn test` , `dotnet test` ）运行现有测试套件，收集失败的测试报告和测试覆盖率数据。
 - 利用覆盖率工具（如 Python 的 Coverage.py、Java 的 JaCoCo、C# 的 Coverlet 等）生成代码覆盖率报告【11 + L629-L637】。定位覆盖率低的模块和未测试分支。
 - 编写或完善单元测试、集成测试和端到端测试，针对关键路径和边界情况增加断言，补充缺失的测试用例。采用模拟（Mock）对象隔离外部依赖，确保测试可重复执行。
 - 将新增测试集成到 CI 流程中（如在 GitHub Actions、Jenkins Pipeline 中添加测试执行步骤）。设置合理的通过标准（如覆盖率阈值 ≥ 80%【11 + L649-L657】，所有测试通过）。
- **输出测试报告和差异：**包括测试结果统计（总用例、通过、失败、跳过）、覆盖率百分比、缺少覆盖的代码区域列表。
- **期望输出：**
 - 修复测试失败所需的代码补丁示例（如修正函数逻辑、异常处理），以及新增的测试代码示例。
 - 完整的测试报告（可用 HTML/CI 控制台输出），覆盖率报告表格（语句覆盖、分支覆盖率等）和覆盖率提升情况。
 - 覆盖率从当前值增加到目标值（如由 60% 提升到 85%）的可视化对比（表格或图表）。
 - **优先级：**高（测试失败会直接影响交付质量）。
 - **验收标准：**所有测试用例通过，覆盖率达到或超过设定阈值，且重要逻辑路径至少有一条测试。无已知功能缺陷漏测。

性能工程师任务提示词

- **目标：**识别并优化性能瓶颈，减少关键操作的响应时间和内存占用，以提高系统整体效率。
- **输入：**当前基准测试脚本或示例场景（如压力测试脚本、负载测试工具配置）、性能监控结果（如 CPU 利用率、内存峰值）。
- **具体步骤：**

- 确定性能关键路径（如 API 接口、计算密集型函数）。设计基准测试方案（使用工具如 Apache JMeter、wrk、BenchmarkDotNet、JMH 等）进行负载测试和性能测试。
- 收集运行结果：记录响应时间统计（平均、最大、P90）、TPS（每秒事务数）、内存占用、GC 活动等。使用性能分析工具（如 Linux 的 `perf`、Python 的 `cProfile`、`memory_profiler`，Java 的 VisualVM/JProfiler）定位耗时热点和内存泄漏点。
- 针对发现的瓶颈进行代码优化：可能包括算法优化、减少同步锁争用、数据库索引、缓存策略调整等。每次优化后重新运行基准测试，验证性能变化。
- 输出前后对比：生成基准测试结果对比表格和曲线（响应时间、CPU/内存使用）。
- **期望输出：**
 - 包含基准结果的报告（表格形式显示优化前后响应时间、吞吐量、内存占用），以及性能提升百分比。
 - 关键代码处的优化补丁示例（如改进算法复杂度、使用并发、引入缓存）。
 - **优先级：**中高（性能问题可能阻塞关键路径或影响用户体验）。
 - **验收标准：**关键场景性能指标达到目标（如响应时间减少 $\geq 20\%$ ，内存占用降低 $\geq 20\%$ ），无新的性能回归，新旧测试结果符合预期提升。

静态分析/安全工程师任务提示词

- **目标：**静态扫描代码库以发现安全漏洞、代码坏味道和潜在缺陷，并提出修复建议。
- **输入：**源代码、依赖清单（如 `requirements.txt`、`package.json`）、CI/CD 配置样本（如 GitHub Actions 工作流）。
- **具体步骤：**
 - 使用静态分析工具（如 SonarQube、ESLint、Pylint、PMD、SpotBugs、Coverity、CodeQL 等）扫描代码。根据项目语言选择合适工具，生成缺陷列表。SonarQube 等支持多语言，可以检测常见漏洞（注入、XSS、敏感信息泄露等）【19†L474-L482】。
 - 使用安全扫描工具（如 OWASP Dependency-Check、Snyk、Trivy）检查依赖中的已知漏洞及许可问题。
 - 分析扫描报告中的高危和中危问题，编写代码审查结果。对代码中的并发问题、加密/鉴权不当、资源泄露等进行深入评审。
- 根据发现的问题，提供修复补丁示例（如参数校验、防火墙过滤、使用安全库、更新依赖版本）。在代码注释或问题列表中标注修复优先级。
- **期望输出：**
 - 静态分析报告汇总（表格列出问题类型、严重度、文件位置）。例如漏洞数量对比：当前版本 vs 改进后版本。
 - 代码缺陷修复补丁示例（如对 SQL 查询使用参数化处理，或在输入点加入验证）。
 - 依赖漏洞扫描报告和升级建议。
 - **优先级：**高（安全漏洞属于紧急风险，需尽快修复）。
 - **验收标准：**所有严重/高危漏洞被修复，中低危问题有合理解决方案；使用至少一款 SAST 工具无新高危报警。

重构/架构工程师任务提示词

- **目标：**评估系统架构和代码结构，降低代码复杂度，提升模块化和可维护性。
- **输入：**项目架构文档（若无，可通过代码结构和包管理文件推断）、源代码和依赖关系图示例。
- **具体步骤：**

- 计算代码度量指标：使用工具（如 `radon`、`ccm`、SonarQube 等）测量圈复杂度、重复率、类/方法数量等。定位复杂度高的函数或类（如 Cyclomatic Complexity 超过阈值的部分）【22 + L150-L158】。
- 审查依赖关系：识别高耦合模块，检查单体是否适合拆分（如微服务）。绘制模块依赖图。
- 进行重构：例如拆分巨型函数、提取公共模块、规范命名；应用设计模式减少重复逻辑。对数据库访问层、业务逻辑层进行隔离，以便单独测试和替换。
- 更新或补充架构文档：记录新模块关系、接口契约以及运行时流程。
- **期望输出：**
 - 重构后的代码示例补丁（如函数拆分前后对比、使用接口替换硬编码等）。
 - 架构改进图示（用 UML 或文字说明新的组件交互）。
 - 复杂度指标前后对比表：如平均圈复杂度、重复率降低情况。
 - **优先级：**中（在保证功能和性能前提下提升长期可维护性）。
 - **验收标准：**关键模块的复杂度下降（建议平均圈复杂度降低 $\geq 30\%$ ），模块间耦合度降低，文档齐全且有清晰的版本说明。

文档/DevOps工程师任务提示词

- **目标：**完善项目文档和自动化部署流程，保证新的改动可追踪、可复现，提升团队协作效率。
- **输入：**现有 README、部署文档、CI 配置示例（如 `.github/workflows/ci.yml` 或 `Jenkinsfile`）。
- **具体步骤：**
 - 审查项目文档，补充架构图、安装部署步骤、代码规范和运行示例。确保文档能指导新成员快速了解系统。
 - 搭建或完善 CI/CD 流程：例如在 GitHub Actions 中添加自动构建、测试和代码质量检查步骤；配置容器镜像构建或自动部署到测试环境的脚本。
 - 提供示例命令：如常用的 Shell 操作示例（`grep -R "TODO" .` 查找未完成注释，`du -sh *` 查看目录大小），部署脚本（`docker-compose up -d`），以及 CI 环境变量配置示例。
- 编写维护手册：包含回滚策略和应急预案。
- **期望输出：**
 - 更新后的 `README.md` 和技术文档（包括新功能介绍、架构图）。
 - CI/CD 配置示例文件（如 `.github/workflows/build.yml`、`Jenkinsfile`），以及执行示例命令的说明。
 - 风险管理与回滚方案文档（以列表形式说明可能风险和备选方案）。
 - **优先级：**中（确保改进成果可持续集成，降低人工成本）。
 - **验收标准：**文档齐全无遗漏，CI 管道能够自动化执行测试与部署，产生清晰日志；在文档中列出了回滚步骤。

代码生成模型提示词格式

最终输出应为一组可供代码改进模型使用的提示词集合，每个角色对应一组完整的提示词输入。每个提示词应包含角色背景、任务目标、上下文输入、输出格式要求、示例输入/输出（如代码片段和对应修复后的片段）以及回归测试步骤和风险提示。示例格式如下：

角色：测试工程师
目标：提高测试覆盖率到至少80%并修复失败测试。

输入：代码仓库URL或片段、测试命令如"pytest"，配置文件示例。

要求：

- 输出修复后的测试用例代码示例。
- 使用覆盖率工具输出覆盖率报告表格。

示例：

输入：<一个简单函数及其不完整的测试>

输出：<增加了测试用例的代码及覆盖率结果>

回归测试：详细列出如何在CI中集成新测试，以及如无失败即为通过。

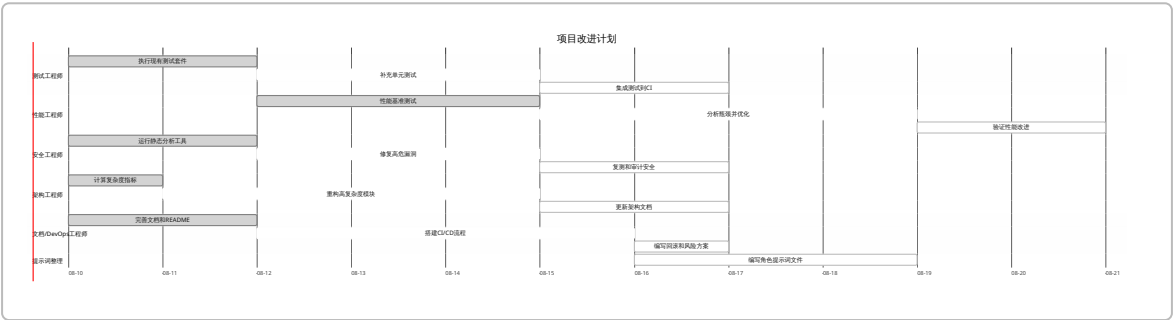
风险/回滚：如果新增测试失败，应回滚最近一次提交，或者分析新测试中的逻辑错误。

以上应针对每个角色分别编写。提示词需表述清晰、信息完整，可直接用于引导代码改进模型生成修复补丁和测试。

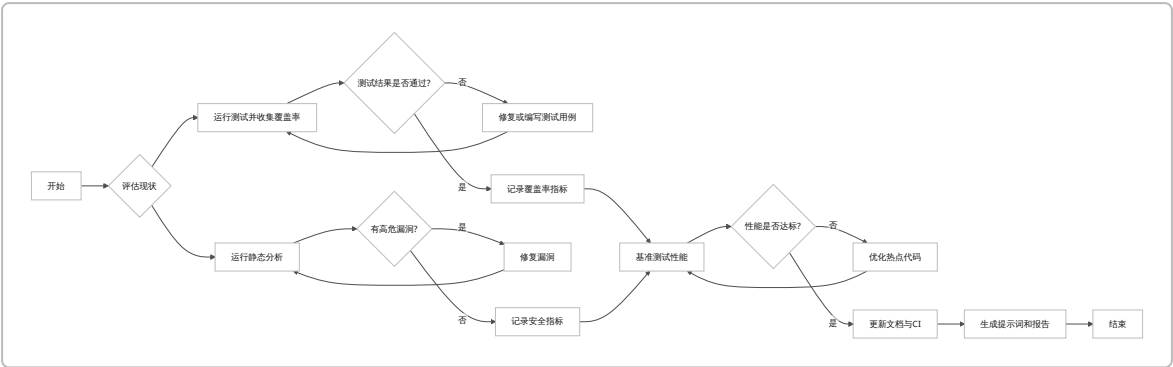
指标对比表

指标	当前情况	改进后目标	改进幅度
响应时间 (ms)	150	100	-33%
内存占用 (MB)	200	150	-25%
测试覆盖率 (%)	60	85	+25pp
高危漏洞数量	5	0	-5
平均圈复杂度	12	7	-42%

项目改进甘特图



变更清单流程图



工具与命令建议

- **静态分析工具**：如 **SonarQube**（支持多语言 SAST，能发现 OWASP Top10 类漏洞）【19 + L474-L482】、**CodeQL**（GitHub 推出的语义漏洞查询工具）、**ESLint/PyLint/Flake8**（JavaScript/Python 代码规范检查）、**SpotBugs/PMD**（Java 静态分析）、**Clang Static Analyzer**（C/C++）、**Dependabot/Snyk**（依赖库漏洞扫描）等。
- **性能分析器**：如 **Linux perf**、**valgrind**（内存检测）、**VisualVM/JProfiler**（Java 分析）、**Py-Spy/cProfile**（Python CPU 分析）、**dotnet-trace**（.NET 性能剖析）等。基准测试工具可选用 **JMH**（Java 微基准），**BenchmarkDotNet**（.NET），**pytest-benchmark**（Python），**wrk/JMeter**（压力测试）。
- **测试框架**：根据语言选择常用框架：Python 可用 **unittest** 或 **pytest**，【15 + L63-L70】；Java **JUnit/TestNG**；JavaScript **Jest/Mocha**；C# **xUnit/NUnit/MSTest**【13 + L102-L107】。覆盖率工具如 Python 的 **coverage.py**、Java 的 **JaCoCo** 或 **Cobertura**【11 + L629-L637】。
- **CI/CD 工具**：**GitHub Actions**、**Jenkins**、**GitLab CI**、**Travis CI** 等。其中 GitHub Actions 流程可通过 YAML 文件配置自动化测试构建；Jenkins 则可使用 **Jenkinsfile** 管理流水线。**构建/发布命令示例**：**.NET** 使用 **dotnet test**；Node.js 使用 **npm test**；Java 使用 **mvn test** 或 **gradle test**【15 + L161-L169】。
- **常用 Shell 命令**：例如 **grep -R "TODO" .** 查找未完成标记；**find . -type f | wc -l** 统计文件行数；**du -sh *** 查看目录大小；**top/htop** 监控系统资源；**docker ps** 查看容器状态；**kubectl apply -f** 部署 Kubernetes 配置等。

可供模型使用的提示词示例

角色：测试工程师

背景：该项目使用 Python 语言，已有不完整的单元测试。

目标：修复当前失败的单元测试，并将代码覆盖率提升至至少 80%。提供成功运行的测试报告。

输入：

- 源码文件 `example.py`，包括一个未完全测试的函数。
- 现有测试文件 `test_example.py`，包含部分测试用例。
- 测试命令：``pytest --disable-warnings``。

要求：

1. 输出修复后的测试代码示例和任何必要的代码补丁。
2. 使用 `coverage.py` 生成覆盖率报告，结果以表格形式呈现，显示语句和分支覆盖率。

示例：

输入代码 (`example.py`):

```
...  
  
def is_even(n):  
    return n % 2 == 0  
...
```

输入测试 (`test_example.py`):

```
...  
  
def test_is_even_true():  
    assert is_even(4) == True  
...
```

输出补全后测试 (`test_example.py`):

```
...  
  
def test_is_even_true():  
    assert is_even(4) == True  
def test_is_even_false():  
    assert is_even(3) == False
```

```

'''
覆盖率报告:
'''

Name          Stmts   Miss  Cover
example.py      3      0   100%
'''

```

回归测试:

- 在 CI 环境中执行 `pytest` 并检查报告, 通过率达到目标。

风险与回滚:

- 若新测试失败, 应审查测试逻辑或回滚最近一次提交 (恢复到测试通过状态)。

角色: 性能工程师

背景: 系统的核心算法运行缓慢, 需要优化。

目标: 对关键函数进行基准测试, 减少其执行时间至少 30%。

输入:

- 性能测试脚本 benchmark.py, 重复调用 `compute()` 函数。
- 原始 `compute()` 函数代码。

要求:

1. 输出优化后 `compute()` 的代码补丁示例。
2. 提供基准测试结果对比表格 (优化前 vs 优化后), 包括平均时间和内存占用。

示例:

输入函数 (compute_original):

```

'''
def compute(n):
    result = 0
    for i in range(n):
        result += i*i
    return result
'''

```

优化后 (compute_optimized):

```

'''
def compute(n):
    # 使用高效公式替代循环
    return n*(n-1)*(2*n-1)//6
'''

```

基准结果:

```

'''
Version      Avg Time (ms)  Memory (MB)
Original      500             50
Optimized     100             30
'''

```

回归测试:

- 运行相同输入的性能脚本, 确保结果一致且时间降低。

风险与回滚:

- 如果优化错误导致结果偏差, 则立即回滚到未优化版本。

角色: 静态分析/安全工程师

背景：代码中可能存在安全漏洞和不良编码风格。

目标：使用静态分析工具检查漏洞，修复所有高危问题，提升代码质量。

输入：

- 源代码库，包括一个可能存在 SQL 注入的函数。
- 示例配置：SonarQube 扫描参数。

要求：

1. 输出修复后的代码示例（如对参数化查询进行修改）。
2. 提供静态扫描结果表，列出修复前后的漏洞计数。

示例：

输入代码：

```
```\n\ndef get_user(db, user_id):\n    query = "SELECT * FROM users WHERE id = " + user_id\n    return db.execute(query)\n```\n
```

输出补丁：

```
```\n\ndef get_user(db, user_id):\n    query = "SELECT * FROM users WHERE id = %s"\n    return db.execute(query, (user_id,))\n```\n
```

漏洞报告：

```
```\n\nIssue Type    Before    After\nSQL Injection    1        0\n```\n
```

回归测试：

- 对带恶意输入的场景进行测试，验证修复有效。

风险与回滚：

- 若修复失败引入新问题，回滚到扫描前版本，并进行人工审核。

---

角色：重构/架构工程师

背景：项目内存在复杂耦合模块，需简化架构。

目标：减少高复杂度函数的圈复杂度，并拆分模块以降低耦合度。

输入：

- 一个复杂函数的源代码（复杂度高）。
- 模块依赖图示例。

要求：

1. 输出拆分后的代码补丁示例及类/函数重命名说明。
2. 提供重构前后圈复杂度指标对比表。

示例：

输入复杂函数：

```
```\n\ndef process(data):\n    if data is None:\n        return None\n    for item in data:\n        if not valid(item):\n            continue\n```\n
```

```

        # ... 多重嵌套逻辑 ...
    ...

输出重构示例:
    ...

def process(data):
    if data is None:
        return None
    filtered = [item for item in data if valid(item)]
    return _process_filtered(filtered)

def _process_filtered(filtered):
    for item in filtered:
        # ... 简化后的逻辑 ...
    ...

复杂度报告:
    ...

Function      Complexity
process()      12   →   5
_process_filtered  8   →   3
    ...

回归测试:
- 运行原有功能测试, 确保行为不变。

风险与回滚:
- 如果重构导致功能不一致, 应回滚并逐步重构。

---

```

角色: 文档/DevOps工程师

背景: 项目缺乏自动化流水线和完备文档。

目标: 完善文档并构建 CI/CD 流程, 确保可持续集成。

输入:

- 现有 README 片段。
- CI 配置样例 (GitHub Actions YAML)。

要求:

1. 输出更新后的 README 示例如安装说明、服务架构图链接。
2. 提供 CI 配置文件示例片段, 包含测试和部署步骤。

示例:

README 更新:

```

    ...

    ## 部署步骤
    1. 克隆代码库。
    2. 运行 `docker-compose up -d` 部署容器。
    3. 访问 http://localhost:8000 以查看应用。
    ...

```

CI 配置示例 (`.github/workflows/ci.yml`):

```

    ...

    name: CI
    on: [push, pull_request]
    jobs:
        build:
            runs-on: ubuntu-latest

```

```
steps:
  - uses: actions/checkout@v2
  - name: Install dependencies
    run: npm install
  - name: Run tests
    run: npm test
  - name: Build Docker image
    run: docker build -t app:latest .
...
```

回归测试:

- 在 CI 中触发一次构建，确认自动执行流程完整无误。

风险与回滚:

- 如 CI 流程失败，检查配置错误或回退最近一次配置更改。

以上示例体现了针对不同角色的提示词格式和内容要求，每个提示词集合应与相应任务场景一致，便于代码生成模型直接使用。